

Software Requirements Specification

for

Motion Detection Surveillance System using Raspberry PI camera module

Version 2.0.0

Prepared by Mikołaj Żyszkiewicz

Lodz University of Technology

20.05.2026

Table of Contents

1. Introduction.....	2
1.1. Purpose	2
1.2. Document Conventions	3
1.3. Intended Audience and Reading Suggestions	3
1.4. Product Scope ----.....	3
1.5. References	3
2. Overall Description.....	3
2.1. Product Perspective	3
2.2. Product Features	3
2.3. User Classes and Characteristics	4
2.4. Operating Environment.....	4
2.5. Design and Implementation Constraints.....	4
2.6. User Documentation.....	4
2.7. Assumptions and Dependencies.....	4
3. System Features.....	4
3.1. «Feature Name».....	4
3.«N». «Additional feature blocks as needed».....	5
4. External Interface Requirements.....	5
4.1. User Interfaces Overview.....	5
4.2. Hardware Interfaces.....	5
4.3. Software Interfaces.....	5
5. System Features/Modules.....	5
5.1. «System Feature Name».....	5

5.1.1 Description and Priority.....	5
5.1.2 Stimulus/Response Sequences.....	5
5.1.3 Functional Requirements -----	6
6. Nonfunctional Requirements ----	6
6.1. Performance	6
6.2. Security	6

Revision History

Name	Date	Reason for Changes	Version
Mikołaj Żyszkiewicz	20.05.2026	Initial version	1.0.0
Mikołaj Żyszkiewicz	02.06.2026	Updated the product features and added a UML graph	2.0.0

1. Introduction

1.1. Purpose

«Identify the product whose software requirements and design are specified in this document, including the revision or release number. Describe the scope of the product that is covered by this specification.»

1.2. Document Conventions

«Describe any standards or typographical conventions that were followed when writing this specification, such as fonts or highlighting that have special significance. For example, state whether priorities for higher-level requirements are assumed to be inherited by detailed requirements, or whether every requirement statement is to have its own priority.»

1.3. Intended Audience and Reading Suggestions

«Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. Describe what the rest of this document contains and how it is organized. Suggest a sequence for reading the document, beginning with the overview sections and proceeding through the sections that are most pertinent to each reader type.»

1.4. Product Scope ----

The software described herein will be used as the firmware for the Raspberry Pi based Motion Detection Surveillance System. The software is designed for autonomous video monitoring of an area in real-time, detecting threats in either interior or restricted areas with no need for ongoing human supervision or reliance upon remote cloud computing infrastructure. The firmware communicates with the camera module installed on the Raspberry Pi to perform environmental telemetry processing and event logging (capture) of digitally recorded evidence only when there are significant visual anomalies. The software performs all computer vision and data management tasks at the network edge to minimize notification time (latency), optimize the use of local storage space, and reduce energy consumption. Therefore, the software offers a secure and cost-effective alternative to commercially available security networks, consistent with the design principles of embedded systems paradigm for stand-alone, edge-computed IoT deployments.

1.5. References

«List any other documents or Web addresses to which this document refers or was based on. These may include user interface style guides, contracts, standards, system requirements specifications, use case documents, or a vision and scope document. Provide enough

information so that the reader could access a copy of each reference, including title, author, version number, date, and source or location. Include the assignment document in the references.»

2. Overall Description

2.1. Product Perspective

The Motion Detection Surveillance System specified in this document is a new, self-contained firmware application engineered to operate entirely at the network edge. It does not serve as a direct follow-on member of an existing product family, nor is it a legacy modification of a pre-existing software suite. Instead, it was conceived as a standalone solution to address distinct architectural vulnerabilities common in contemporary commercial security infrastructures.

Historically, localized surveillance has depended heavily on continuous human oversight or high-bandwidth cloud video recording (CVR) pipelines. These dependencies introduce single points of failure, including high operational subscription costs, severe latency bottlenecks, and significant data privacy risks. This product originates from the engineering need to decouple real-time visual threat detection from external networks. By shifting the computational workload to a low-cost, edge-computed hardware platform—specifically the Raspberry Pi architecture—the system serves as a decentralized replacement for traditional, cloud-dependent IP camera systems.

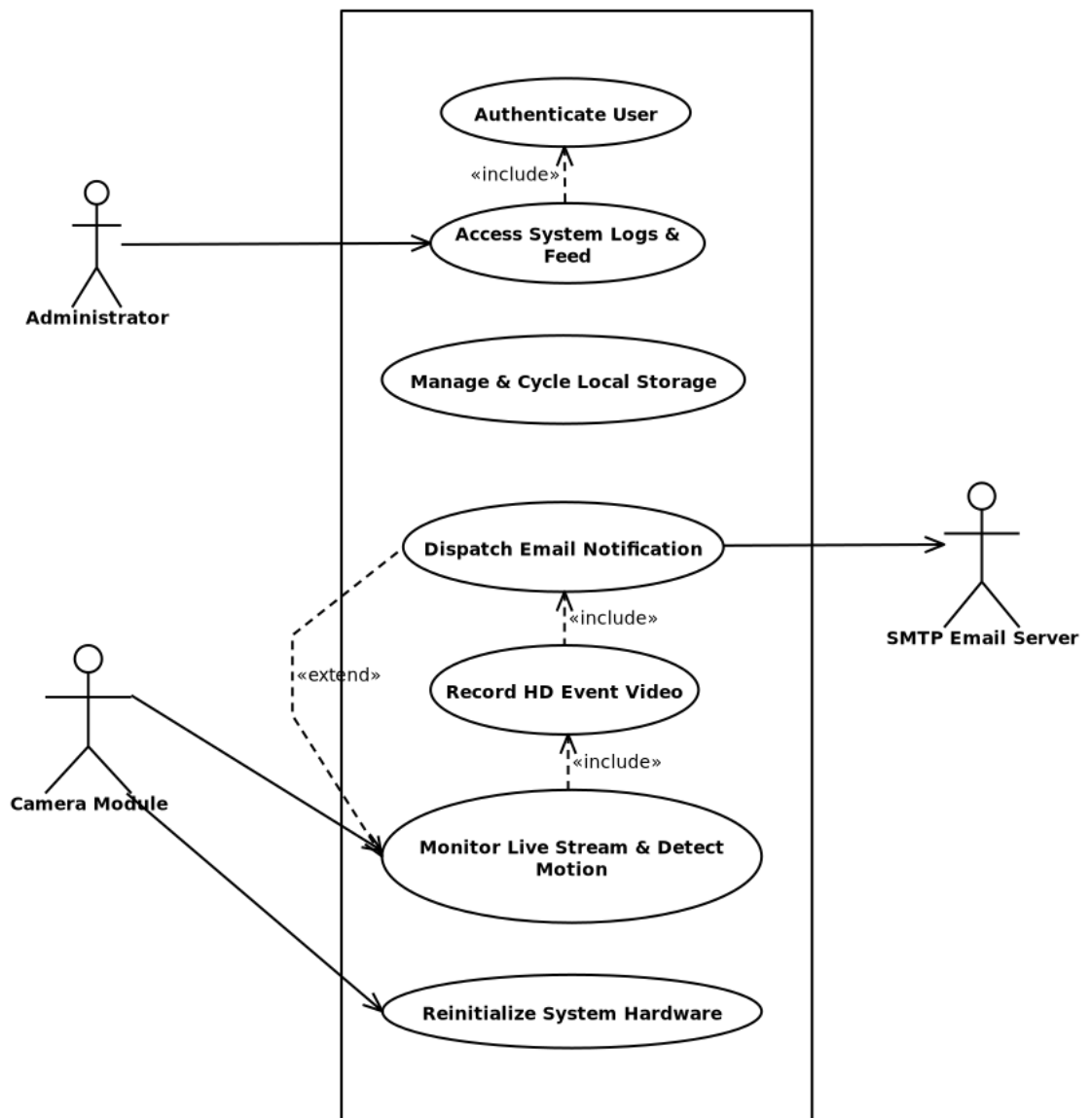
As a self-contained entity, the firmware manages the entire lifecycle of a security event locally. It directly controls the hardware abstraction layer of the camera module, processes the video stream frame-by-frame, and manages its own localized storage and alerting protocols. This independent design ensures that the system remains fully operational even during network isolation, establishing a secure, privacy-focused foundation for localized Internet-of-Things (IoT) security deployments.

2.2. Product Features

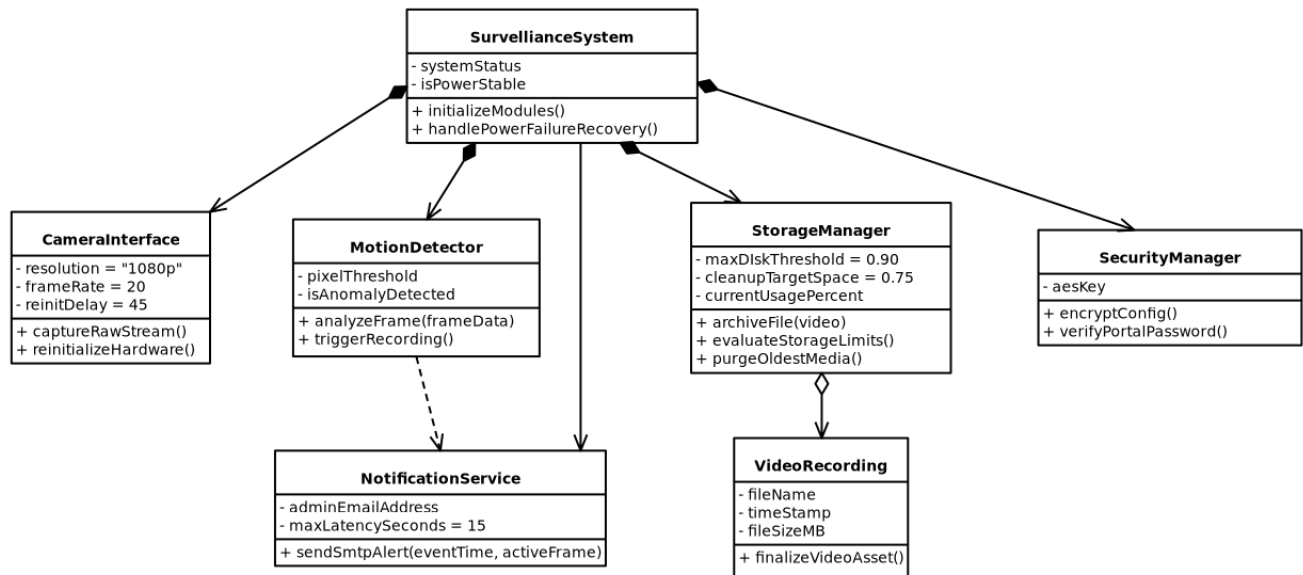
- **Real-Time Motion Detection:** Continuously processes the raw video stream frame-by-frame at the edge to detect localized visual anomalies.
- **Automated Video Recording:** Initiates high-definition local recording (1080p at 20fps) for a fixed window whenever motion thresholds are crossed.
- **Decentralized Media Management:** Automatically formats, labels, and archives video files locally, with built-in rolling storage cleanup when disk usage peaks.

- **Instant Alert & Telemetry Dispatch:** Generates and distributes secure email alerts complete with precise timestamps and evidence images directly to an administrator.
- **Edge Data Security & Authentication:** Restricts system configuration access via hashed credentials and encrypts sensitive network keys locally.
- **Fault-Tolerant Power Recovery:** Self-heals and automatically reinitializes critical hardware components following electrical power disruptions.

uc Motion Detection Surveillance System



The internal firmware architecture of the Raspberry Pi Motion Detection Surveillance System is strictly object-oriented to ensure modularity and ease of maintenance. The system utilizes dedicated software interfaces to standardize edge device orchestration, while strictly separating the volatile, real-time frame processing and computer vision logic from the non-volatile local disk storage and rolling media management mechanics. The UML Class Diagram below illustrates the composition of the main surveillance controller, its tight encapsulation of peripheral hardware modules, and its operational media asset aggregation methods.



2.3. User Classes and Characteristics

«Identify the various user classes (intended target for the product) that you anticipate will use this product. User classes may be differentiated based on frequency of use, subset of product functions used, technical expertise, security or privilege levels, educational level, or experience. Describe the pertinent characteristics of each user class. Certain requirements may pertain only to certain user classes. Distinguish the most important user classes for this product from those who are less important to satisfy.»

2.4. Operating Environment

«Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.»

2.5. Design and Implementation Constraints

«Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).»

2.6. User Documentation

«List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.»

2.7. Assumptions and Dependencies

«List any assumed factors (as opposed to known facts) that could affect the requirements stated in the specification. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).»

3. System Features

«The purpose here is to give a brief introduction to the system features, so that §4 has something to reference. Then, the full explanation of the system features is given in §5, likely referencing §4. The numbering in §3 and §5 should match.»

3.1. «Feature Name»

«Brief description of the feature, generally 1-3 sentences»

3.«N». «Additional feature blocks as needed»

4. External Interface Requirements

4.1. User Interfaces Overview

«Describe the high level functionality of the system from the user's perspective. Explain how the user should be able to use your system to complete all the expected features and the feedback information that will be displayed for the user (for each screen the user will see). For example, specify whether it is GUI or text based interface and how at high level it would work. Discuss at high level how user will interact with the game such as clicking buttons, typing in some selection character, etc.»

«Describe the minimum requirements for the interface. This may include some sample screen images whether for text or GUI approach, any GUI standards or product family style guides that are to be followed, screen layout constraints, for GUI standard buttons and functions (e.g., help) that must appear on every screen, keyboard shortcuts, error message display standards, and so on»

4.2. Hardware Interfaces

«Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.»

4.3. Software Interfaces

«Describe the connections between this product and other specific software components (name and version), including databases, interpreters, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.»

5. System Features/Modules

«This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product.»

5.1. «System Feature Name»

5.1.1 Description and Priority

«Provide a short description of the feature. Give its priority, too.»

5.1.2 Stimulus/Response Sequences

«"Stimulus" or "Condition"»: «Describe the stimulus or condition»

Response: «Describe the response»

«Additional Stimulus/Response pairs as needed»

5.1.3 Functional Requirements -----

REQ-1.1: Whenever the System detects a 5% difference in pixels between two consecutive frames, the System should start recording a 1080p video with 20fps for a period of 15 seconds.

REQ-1.2: Once the 15 second recording time is completed, the System should save the recorded video in the local folder using the format YYYYMMDD_HHMMSS.mp4.

REQ-1.3: After successfully saving a new video, the System should automatically send an e-mail notification containing a timestamp of the captured motion event along with a JPEG image of the most active frame to the configured administrator address within 10 seconds.

REQ-1.4: Once the local storage usage exceeds 90%, the System should remove oldest media files sequentially until the disk space usage drops below 75%. REQ-1.5: Once a system power failure occurs, the System will reinitialize the camera module interface once it becomes ready after 45 seconds.

5.«N». «Additional feature blocks»

6. Nonfunctional Requirements ----

6.1. Performance

NF-1.1: The motion detection engine should process incoming raw video stream coming from the Raspberry Pi

NF-1.2: The end to end latency from the moment physical motion was detected until its final delivery to the configured administrator address in an email should not exceed 15 seconds.

6.2. Security

NF-2.1: Configuration payloads such as SMTP passwords, Wi-Fi keys, and administrator email parameters shall be securely stored in non-volatile memory using AES-256 encryption.

NF-2.2: User authentication using SHA-256 hashing algorithm for password validation shall be mandatory for the web portal through which system logs and live streaming feeds are accessed.

6.3. Reliability

NF-3.1: 99.5% uptime shall be achieved by the system during any test period of 30 days except the time taken for site-wide electrical infrastructure maintenance.